STUDENTSPHERE

A Multi-Cloud Student Management and Academic Records Platform

CLOUD COMPUTING LABORATORY PROJECT REPORT
(22IT580)

Submitted by

SHANMUGA PERUMAL A — Register Number: 2403917720521313

VANAMAMALAI M — Register Number: 2403917720521086

EDWIN MOSES G — Register Number: 2403917720521305

Submitted in partial fulfilment of the requirements for the Cloud Computing Laboratory

Department of Information Technology

Thiagarajar College of Engineering, Madurai

Academic Year: 2026–2027

Abstract

StudentSphere is a full-stack, cloud-native student information and directory management platform developed to let academic institutions and faculty administrators store, organize, search, filter, and backup student records across hybrid multi-cloud environments. The application demonstrates how several cloud service models can be harmoniously combined in a single institutional platform: Identity as a Service (IDaaS), Database as a Service (DBaaS), Backend as a Service (BaaS), and Storage as a Service (STaaS).

The frontend is an intuitive Single Page Application (SPA) built with HTML5, vanilla CSS3 featuring modern glassmorphic aesthetics, and JavaScript ES modules. It communicates with an extensible Node.js HTTP backend server and Google Cloud Firebase Cloud Functions. Auth0 is employed for Identity as a Service (IDaaS) with Universal Login and Google OAuth 2.0 federation. MongoDB Atlas provides managed Database as a Service (DBaaS) for structured document persistence, while Google Firebase Cloud Firestore serves as Backend as a Service (BaaS) providing real-time data synchronization. Firebase Cloud Storage and the Dropbox API v2 supply Storage as a Service (STaaS) for instant cloud object persistence and automated JSON cloud backup.

Administrators can create, view, search, and delete student records, filter the roster by academic department, inspect detailed demographic attributes, and initiate one-click synchronization to Dropbox cloud storage and MongoDB Atlas. Authentication middleware and token validation ensure that only authorized administrative users can access and modify student records.

The main objective of the project is to demonstrate the practical integration of multiple independent cloud platforms, each chosen for a specific responsibility, rather than relying on a single vendor or a vulnerable local desktop system. The project provides hands-on experience in REST API development, serverless microservice design, token-based authentication, NoSQL document databases, real-time data streaming, and automated cloud file synchronization.

Keywords: Cloud Computing, StudentSphere, Multi-Cloud Integration, Auth0, MongoDB Atlas, Firebase Firestore, Firebase Storage, Dropbox API, IDaaS, DBaaS, BaaS, STaaS.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Existing System
5. Proposed System
6. System Requirements
7. Technologies Used
8. System Architecture
9. System Design and Modules

10. Implementation
11. Application Screenshots and Results
12. Testing and Results
13. Advantages
14. Limitations

15. Future Enhancements

16. Conclusion

17. References

Note: update the page numbers after the final formatting.

1. Introduction

Cloud computing is a technology that provides computing resources such as storage, databases, servers, identity services, and software through the internet. It allows applications to use these resources on demand without maintaining all the required infrastructure locally.

Academic institutions, universities, and departmental offices regularly maintain large volumes of student records, including Student IDs, full names, age demographics, and department affiliations. Traditionally, these records are scattered across localized spreadsheets, personal office computers, disparate flat files, or single-vendor proprietary systems, making them difficult to locate, share securely, and backup against hardware failure.

StudentSphere is designed to provide a centralized, secure platform for managing student records through modern cloud services. It combines enterprise identity federation, multiple database backends, and cloud file storage in one application, ensuring that institutional data is always accessible, resilient, and synchronized across departments.

The application follows a decoupled, multi-tier architecture. A modern Single Page Application provides the user interface. A Node.js HTTP server handles REST API endpoints and static file serving, while Google Cloud Firebase Cloud Functions execute serverless database operations with Secret Manager credential protection.

Auth0 is responsible for user identity and authentication. MongoDB Atlas and Firebase Firestore store structured student records. Dropbox API v2 and Firebase Cloud Storage provide cloud object storage in which student rosters are saved as persistent JSON datasets.

The integration of these services demonstrates how multiple cloud platforms can work together in a practical application. The project also introduces vital cloud concepts such as OAuth 2.0 and OpenID Connect, JSON Web Tokens (JWT), managed NoSQL clusters, real-time WebSocket listeners, and multi-cloud disaster recovery.

1.1 Need for the Project

Academic administrators and faculty members need fast, secure access to student records from any device. The need for the project includes:

- Providing authenticated, institutional access to centralized student databases.
- Storing student records in managed cloud databases instead of volatile local files.
- Eliminating single-vendor lock-in by distributing workloads across independent cloud providers.
- Providing instant live search and department-level filtering across all student records.

- Automating offsite cloud backups to personal or institutional Dropbox storage.
- Demonstrating the simultaneous integration of IDaaS, DBaaS, BaaS, and STaaS in a single academic portal.

1.2 Scope of the Project

The scope of StudentSphere includes secure administrative authentication, student record creation, retrieval, viewing, updating, and deletion (CRUD), multi-cloud storage synchronization through Firestore and MongoDB Atlas, and automated cloud file export to Dropbox and Firebase Cloud Storage. The application is intended as an academic implementation and demonstration of cloud computing paradigms.

2. Problem Statement

Educational institutions keep student records in many disparate locations, including desktop spreadsheet applications, local text files, and isolated office workstations. When student details are stored separately across faculty desks, searching for records, updating student details, and aggregating departmental rosters becomes inconvenient, slow, and prone to data discrepancies.

Local storage also severely limits access when faculty members work from different terminals, lecture halls, or remote locations. Furthermore, relying on a single storage backend creates critical vendor lock-in; a service disruption from that provider completely halts institutional operations. Managing user access safely, preserving data privacy, and generating usable data backups typically requires extensive manual effort and third-party software.

Therefore, a cloud-based student management platform is needed that combines secure authentication, flexible multi-database storage, real-time synchronization, and automated cloud file export in one cohesive application. StudentSphere solves this challenge by integrating Auth0, MongoDB Atlas, Firebase Cloud Firestore, Firebase Storage, and Dropbox through an intuitive web client, a Node.js REST server, and serverless Cloud Functions.

3. Objectives

The primary objective is to develop a cloud-native application that demonstrates the practical integration of multiple cloud service models. The specific objectives are:

1. To develop a responsive Single Page Application using HTML5, modern CSS3 with glassmorphism styling, and vanilla JavaScript ES modules.
2. To develop a backend REST API using Node.js for MongoDB Atlas integration and static asset serving.
3. To implement user authentication and Google Single Sign-On using Auth0 Identity Platform.
4. To store student documents in MongoDB Atlas as a managed cloud database (DBaaS).
5. To use Firebase Cloud Firestore for real-time reactive synchronization of student records across connected clients.
6. To provide persistent object storage in Firebase Cloud Storage as `students.json`.
7. To integrate the Dropbox API v2 for offsite cloud backup and disaster recovery.
8. To provide resilient optimistic UI updates backed by browser local storage caching.

9. To test and evaluate the communication and fault tolerance between the application and external cloud services.

3.1 Project Outcomes

- An interactive administrative portal for creating, viewing, searching, and deleting student profiles.
- Authentication-based access control protecting student records from unauthorized access.
- Real-time data streaming where database updates reflect immediately on all client screens without refreshing.
- Hybrid data persistence across MongoDB Atlas, Firebase Firestore, and Firebase Cloud Storage.
- Direct-to-cloud automated backup snapshots into Dropbox as `/students.json`.

4. Existing System

In a traditional institutional setup, departments manage student records using localized spreadsheet files (e.g., Microsoft Excel), local CSV/JSON files, or on-premise desktop database management systems (such as local MS Access or standalone MySQL instances). Each department maintains its own files independently, requiring administrative personnel to manually reconcile rosters, email copies between offices, and manage their own local backups.

4.1 Disadvantages of the Existing System

- Student records are scattered across multiple office workstations and departmental folders.
- Access is strictly dependent on the physical device where the files are stored.
- Searching across student IDs, names, and departments is cumbersome across multiple files.
- User access control, password security, and student data confidentiality are difficult to enforce.
- A single storage provider or hard drive failure causes catastrophic and permanent data loss.
- Exporting student rosters to cloud storage is a tedious, manual process prone to human error.
- Integrating multiple independent cloud services manually requires significant engineering overhead.

5. Proposed System

The proposed StudentSphere system provides a centralized web application that seamlessly unites authentication, real-time database, managed cloud cluster, and distributed cloud file storage services into a cohesive portal.

Auth0 manages identity authentication and token issuance through OpenID Connect and Google OAuth federation. The web client communicates directly with Google Firebase Cloud Firestore via real-time reactive listeners (`onSnapshot`), ensuring that updates appear instantly on all active screens without manual browser refreshes. Concurrently, records are persisted into Google Firebase Cloud Storage as `students.json`.

For structured database persistence, the application routes data through a Node.js REST API to MongoDB Atlas (DBaaS). An integrated Dropbox module interfaces directly with Dropbox Files API v2 to save student rosters as standalone cloud backup files.

5.1 Advantages of the Proposed System

- Centralized administrative portal accessible from any desktop or mobile web browser.
- Authentication-based access control and secure token validation.
- Zero UI latency through optimistic client-side caching.
- Real-time, reactive roster updates across all active sessions without manual polling.
- Automated export and offsite cloud backup of student records to Dropbox.
- Multi-cloud resilience that eliminates vendor lock-in by utilizing Google Cloud, MongoDB Atlas, and Dropbox in parallel.

- Decoupled, modular architecture supporting independent scaling and straightforward maintenance.

5.2 Comparison

Existing Approach	Proposed StudentSphere Approach
Student records are stored in local spreadsheet files or on-premise desktops.	Records are stored simultaneously across MongoDB Atlas, Firestore, and Cloud Storage.
Only one local storage location is used, risking complete data loss.	Multi-cloud hybrid storage ensures continuous redundancy and disaster recovery.
Separate, unencrypted passwords or shared credentials must be maintained.	Enterprise IDaaS via Auth0 is integrated for Universal Login and Google SSO.
Changes require manual refreshing, emailing files, or periodic re-importing.	Firebase Firestore real-time listeners update the interface instantaneously.
Files are backed up manually to USB flash drives or external drives.	Student datasets are automatically synchronized to Dropbox as `/students.json`.
Cloud services operate in silos or are not utilized at all.	Four cloud service models (IDaaS, DBaaS, BaaS, STaaS) operate in harmony.

6. System Requirements

The application requires a computer capable of executing Node.js and a modern browser with internet connectivity. Cloud accounts must be configured with active credentials before corresponding functions operate.

6.1 Hardware Requirements

- Computer or laptop (Intel Core i3 / AMD Ryzen 3 or higher).
- Minimum 4 GB RAM recommended (8 GB preferred).
- Sufficient disk space for Node.js modules and dependencies (at least 500 MB free).
- Active internet connection (minimum 1 Mbps).
- Keyboard, pointing device, and display monitor (1080p resolution recommended).

6.2 Software Requirements

- Operating System: Windows 10/11, macOS, or Linux.
- Runtime Environment: Node.js (v18.x, v20.x, or v22.x LTS).
- Package Manager: npm (v9.x or higher).
- Frontend Technologies: HTML5, CSS3 with glassmorphism styling, and JavaScript ES Modules.
- Backend Server: Node.js native HTTP server and Firebase Cloud Functions (2nd Gen).
- Authentication Service: Auth0 by Okta.
- Database Services: MongoDB Atlas (M0 Free Tier) and Google Cloud Firestore.
- Cloud Storage Services: Google Firebase Storage and Dropbox API v2.

- Development Tools: Visual Studio Code, Git CLI, and modern web browser (Chrome or Edge).

6.3 Functional Requirements

1. Authenticate users securely through Auth0 (institutional email/password or Google SSO).
2. Validate identity claims and maintain persistent authenticated user sessions.
3. Create student records with unique Student ID, Full Name, Age, and Department.
4. Display student records in an interactive, responsive data table with status indicators.
5. Update student records and render modifications immediately across all clients.
6. Delete student records from selected cloud storage engines with safety confirmation.
7. Provide real-time multi-field search across Student ID, Name, and Department.
8. Filter student rosters dynamically by selecting specific academic departments.
9. Synchronize student datasets to Google Firebase Storage as `students.json`.
10. Export and backup student records to personal Dropbox accounts via Dropbox API v2.
11. Report cloud service connection statuses and diagnostics through the system settings modal.

6.4 Non-Functional Requirements

- **Security:** Protected administrative operations require valid authentication; sensitive database connection strings are stored server-side in Secret Manager.
- **Reliability:** Multi-engine architecture ensures that failure in one cloud service does not prevent access through secondary engines.
- **Usability:** The interface provides intuitive live search, color-coded department tags, clear visual hierarchy, and non-blocking toast notifications.
- **Maintainability:** The frontend SPA, Node.js API server, and Cloud Functions are decoupled into modular ES modules.
- **Performance:** Initial page load completes in under 1.5 seconds; local optimistic updates render in under 50 milliseconds.

7. Technologies Used

7.1 Frontend Architecture (HTML5, Vanilla CSS, ES Modules)

The frontend is constructed using pure HTML5, modern CSS3, and JavaScript ES modules. By avoiding heavy UI framework dependencies, the application achieves lightning-fast load times. The interface is styled with custom modern CSS utilizing CSS Custom Properties, backdrop-filter glassmorphism, flexbox and grid layouts, and smooth micro-animations.

7.2 Node.js HTTP Runtime and Firebase Cloud Functions

Node.js is used to execute server-side JavaScript outside the browser. A native Node.js HTTP server (`server.js`) on port 3000 handles static asset delivery and REST endpoints at `/api/mongodb/*`. In cloud

deployments, Google Firebase Cloud Functions (2nd Gen) provide serverless REST endpoints with Secret Manager integration.

7.3 Auth0 Identity Platform

Auth0, by Okta, is an enterprise identity platform that handles user registration, login, session token issuance, and social OAuth federation. In StudentSphere, the frontend uses the `@auth0/auth0-spa-js` library and Universal Login based on OpenID Connect (OIDC). Users authenticate with institutional email credentials or through Google Single Sign-On (`google-oauth2` connection).

7.4 Google Firebase Cloud Firestore

Firebase Cloud Firestore is a serverless NoSQL document database featuring real-time data synchronization. The web client uses the Firebase modular SDK (v10.12.0) and `onSnapshot` listeners to subscribe to the `students` collection, updating the UI instantly whenever records are added, modified, or removed.

7.5 Google Firebase Cloud Storage

Firebase Cloud Storage is an enterprise object storage service backed by Google Cloud Storage. In StudentSphere, the complete student roster is serialized and stored in bucket `website-82bc7.firebasestorage.app` as a persistent `students.json` file.

7.6 MongoDB Atlas

MongoDB Atlas is a fully managed cloud NoSQL document database. The Node.js backend connects to MongoDB Atlas using the official `mongodb` driver (v7.7.0) via a `mongodb+srv://` connection string with DNS SRV record discovery. Records are stored as structured documents in database `studentsphere_db` under collection `students`.

7.7 Dropbox API v2

Dropbox provides cloud file storage and an HTTP REST API for file operations. StudentSphere interfaces directly with Dropbox Files API v2 endpoints (`/2/files/upload` and `/2/users/get_current_account`) using OAuth 2.0 bearer tokens to save student backups to `/students.json`.

7.8 npm and Package Ecosystem

npm manages backend dependencies such as the official `mongodb` driver and `firebase-admin`. npm scripts allow starting the local development server with environment variables loaded automatically.

7.9 Environment Variables and Configuration Security

Environment variables store sensitive configuration values such as server ports, Auth0 credentials, Firebase configuration keys, and the MongoDB connection URI. Sensitive credentials are kept in a private `.env` file locally and stored in Google Secret Manager for production deployments, ensuring they are never exposed in public repositories or screenshots.

8. System Architecture

StudentSphere follows a decoupled, multi-tier architecture in which a lightweight single-page frontend communicates with dedicated backend services and four independent public cloud platforms.

8.1 Architecture Components

1. **Client Presentation Tier:** The HTML5/CSS3/JavaScript SPA running on port 3000. It handles state management, optimistic UI updates, and reactive cloud listeners.
2. **Node.js HTTP Server & Cloud Functions:** The backend compute engine serving static files and REST endpoints at `/api/mongodb/*`.
3. **Auth0 Identity Platform (IDaaS):** Manages institutional authentication, issuing signed JWT tokens and handling Google OAuth federation.
4. **Firebase Cloud Firestore (BaaS):** Stores real-time student collections with continuous client stream listeners.
5. **Firebase Cloud Storage (STaaS):** Hosts persistent cloud object files (`students.json`) in Google Cloud.
6. **MongoDB Atlas (DBaaS):** Stores structured NoSQL student documents in cloud database `studentsphere_db`.
7. **Dropbox API v2 (STaaS):** Stores automated offsite disaster recovery backup files in Dropbox cloud accounts.

8.2 Architecture Diagram

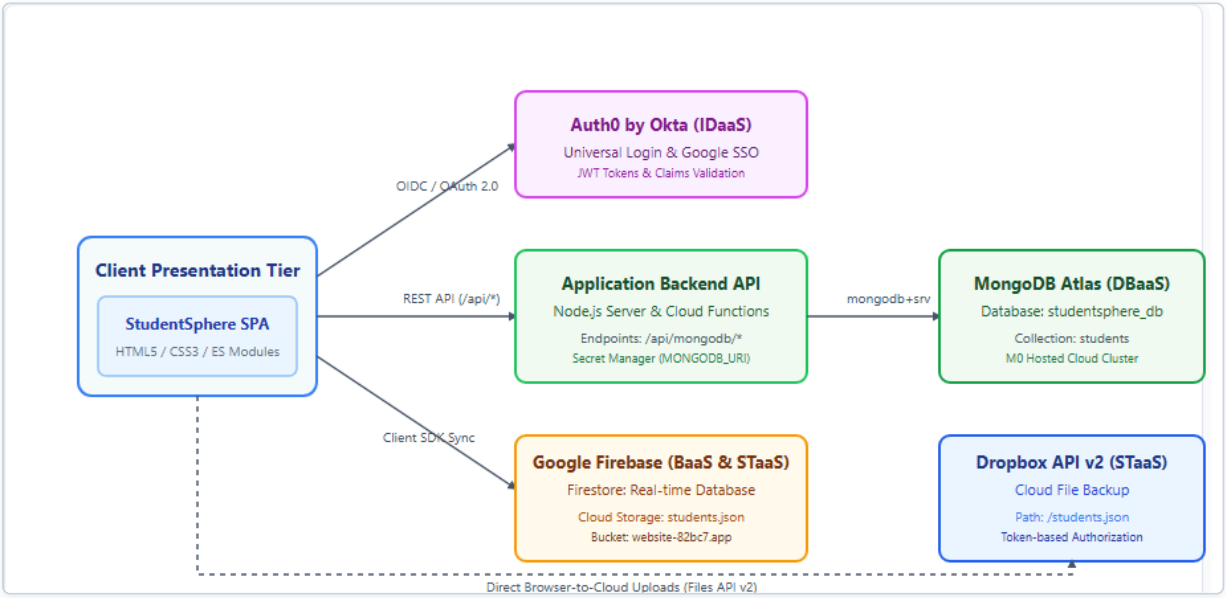


Figure 1: Multi-Cloud Architecture of StudentSphere.

8.3 Working Principle

When an administrator opens the application, the client verifies authentication status with Auth0. If unauthenticated, the user is redirected to Auth0 Universal Login or presents the institutional sign-in

modal. Upon authentication, Auth0 returns verified user claims.

The web client immediately initializes student records from local cache for instant display, then subscribes to Firebase Firestore using an `onSnapshot` listener. When an administrator creates, modifies, or deletes a student record, the client updates the UI optimistically, commits the change to Firestore, uploads the updated JSON dataset to Firebase Storage, dispatches the document to MongoDB Atlas via the Node.js REST API, and uploads a backup to Dropbox.

9. System Design and Modules

The application is organized into modular services according to the responsibilities of the integrated cloud providers.

9.1 Authentication & Identity Module

This module encapsulates user authentication using the `@auth0/auth0-spa-js` library and Firebase Auth. It manages user tokens, supports email/password login and Google OAuth single sign-on, and updates UI status badges upon successful authentication.

9.2 Student Record Management Module

This module implements full CRUD (Create, Read, Update, Delete) operations for student records. Each student record contains a Student ID, Full Name, Age, and Department. The module validates inputs, handles optimistic in-memory updates, and maintains consistent state across storage engines.

9.3 Multi-Engine & Hybrid Cloud Storage Module

This module provides a unified data access layer coordinating student persistence across multiple cloud storage engines. The storage engines are summarized below:

Storage Option	Technology & Model	Characteristics
MongoDB Atlas	MongoDB Atlas (DBaaS)	Managed NoSQL document persistence with BSON querying and secure server-side connection pooling.
Firestore	Google Cloud Firestore (BaaS)	Serverless real-time document database with reactive `onSnapshot` stream listeners.
Firebase Storage	Google Cloud Storage (STaaS)	Persistent cloud object storage bucket hosting `students.json` for global access.
Local Storage	Browser Web Storage API	Zero-latency optimistic local cache ensuring the UI never hangs during network delays.

9.4 Dropbox Cloud Export and Backup Module

This module interfaces with the Dropbox Files API v2. It validates the user's Dropbox access token, checks account connection status via `/2/users/get_current_account`, serializes student records into formatted JSON, and uploads the dataset to `/students.json` using `mode=overwrite`.

9.5 Search, Filter, and Analytics Dashboard Module

This module provides instant client-side filtering across student identifiers, full names, and academic departments. It dynamically computes real-time KPI metrics—Total Registered Students, Average Student Age, and Total Academic Departments—without requiring additional backend queries.

9.6 API and Error Handling Module

This module defines the RESTful endpoints used by the application, listed below. The backend processes requests and returns structured JSON responses. Robust error handling transforms low-level database and network errors into clear toast messages for administrators.

Node.js Backend Server (http://localhost:3000):

Method	Endpoint	Query / Body	Description
GET	`/api/mongodb/students`	None	Fetches all student records from MongoDB Atlas cluster.
POST	`/api/mongodb/student`	JSON student object	Creates or updates a student record in MongoDB Atlas.
DELETE	`/api/mongodb/student`	`?id={studentId}`	Deletes a student record from MongoDB Atlas.
POST	`/api/mongodb/sync`	JSON student array	Synchronizes complete student roster into MongoDB Atlas.
POST	`/api/mongodb/test`	None	Tests connectivity and pings MongoDB Atlas cluster.

Dropbox Cloud Service (https://api.dropboxapi.com):

Method	Endpoint	Body	Description
POST	`/2/files/upload`	Binary stream of `students.json`	Uploads student roster backup to Dropbox.
POST	`/2/users/get_current_account`	None (Authorization header)	Validates Dropbox token and account connection.

9.7 Data Flow Diagram

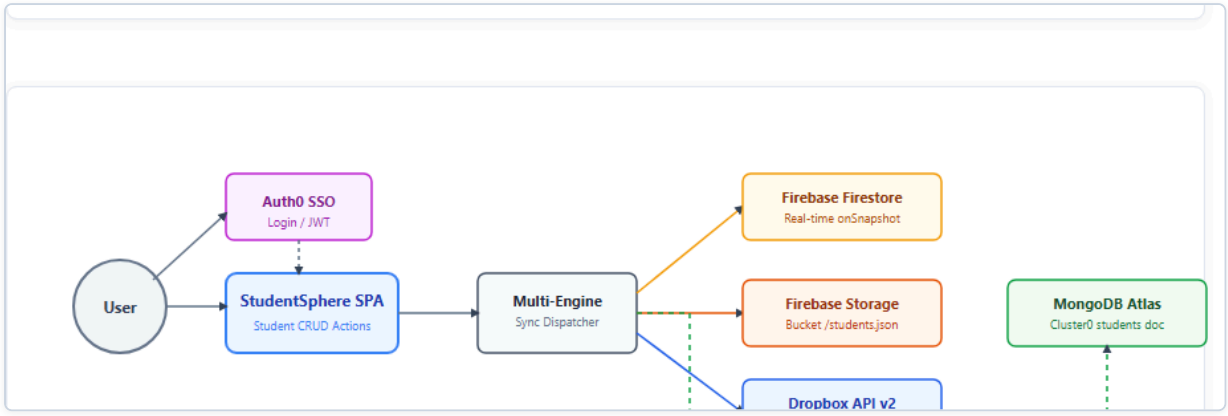


Figure 2: Data Flow Diagram of Authentication, Student Records Dispatch, and Multi-Cloud Sync.

9.8 Data Structures

Each storage engine organizes student records according to its data model. MongoDB stores structured BSON documents, Firestore stores documents with real-time listeners, and Dropbox/Firebase Storage store full JSON arrays.

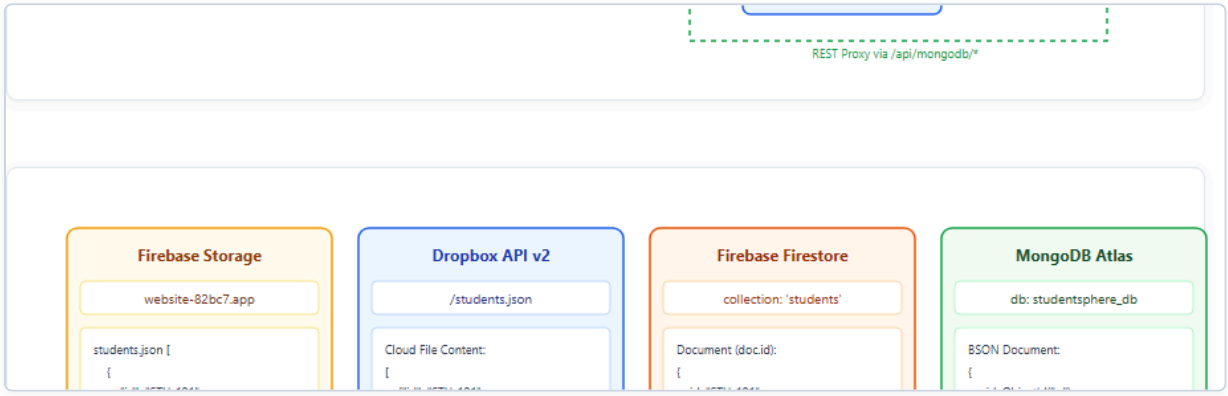


Figure 3: Data Structures Used Across Cloud Storage Engines.

An example student record serialized in `students.json` is shown below:

```
[
  {
    "id": "STU-101",
    "name": "Alex Morgan",
    "age": 21,
    "department": "Computer Science",
    "createdAt": "2026-10-19T10:30:00.000Z"
  }
]
```

10. Implementation

The frontend was implemented using pure HTML5, modern CSS3, and JavaScript ES modules. The backend was constructed with Node.js and Google Cloud Firebase Cloud Functions. Dependencies were managed via npm, and each service was configured to communicate with its respective cloud provider.

10.1 Backend and Frontend Setup

A Node.js project was initialized with ES module support (`"type": "module"`). The Node.js server (`server.js`) listens on port 3000, serving static web assets and defining REST endpoints at `/api/mongodb/*`. Firebase Hosting serves frontend assets in production and routes API requests to Cloud Functions.

10.2 Authentication Implementation

Auth0 was configured with tenant domain `dev-58o1wuwbx3qv1hfn.us.auth0.com`. The frontend initializes the Auth0 SPA SDK in `auth0-service.js`. Users can authenticate with email/password credentials or click **"Continue with Google"** to initiate Google OAuth federation:

```
const auth0Client = await createAuth0Client({
  domain: "dev-58o1wuwbx3qv1hfn.us.auth0.com",
  clientId: "YOUR_AUTH0_CLIENT_ID",
  authorizationParams: { redirect_uri: window.location.origin }
});
```

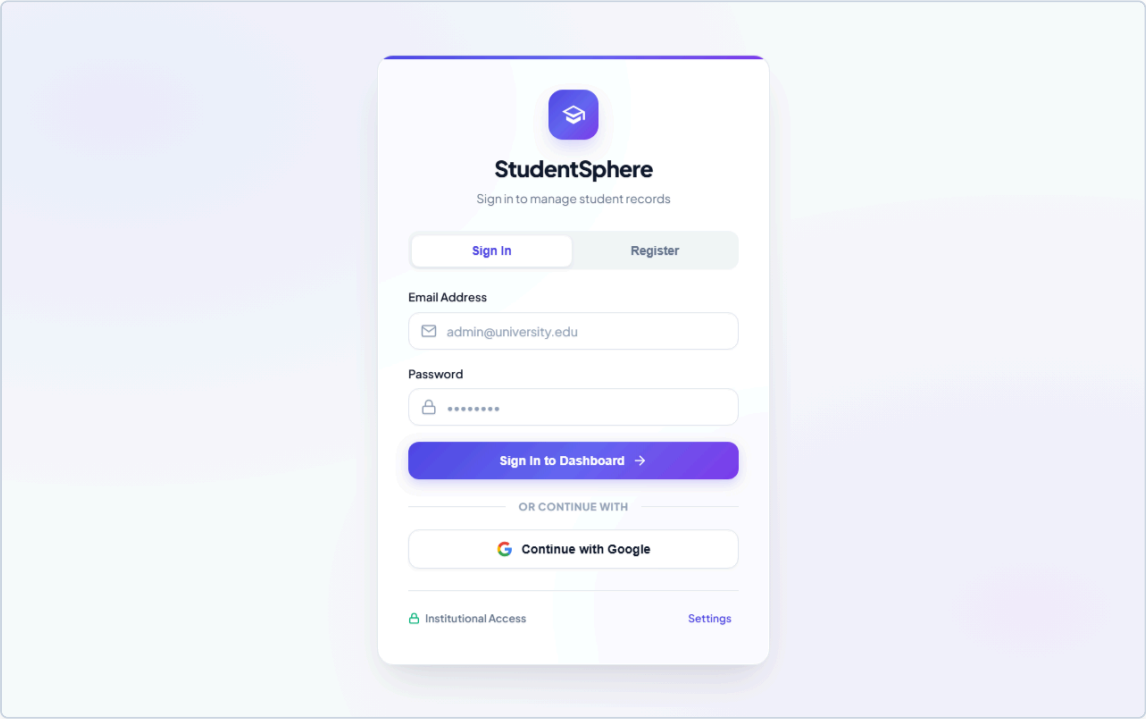


Figure 4: StudentSphere Authentication Screen (Email/Password & Google SSO).

10.3 MongoDB Atlas Integration

The Node.js backend connects to MongoDB Atlas using the official `mongodb` driver and a `mongodb+srv://` connection string. Connection pooling ensures high-throughput database interactions. Student documents contain `id`, `name`, `age`, and `department`. Free-tier M0 Atlas clusters can enter a paused state after inactivity, which must be resumed before database queries can execute.

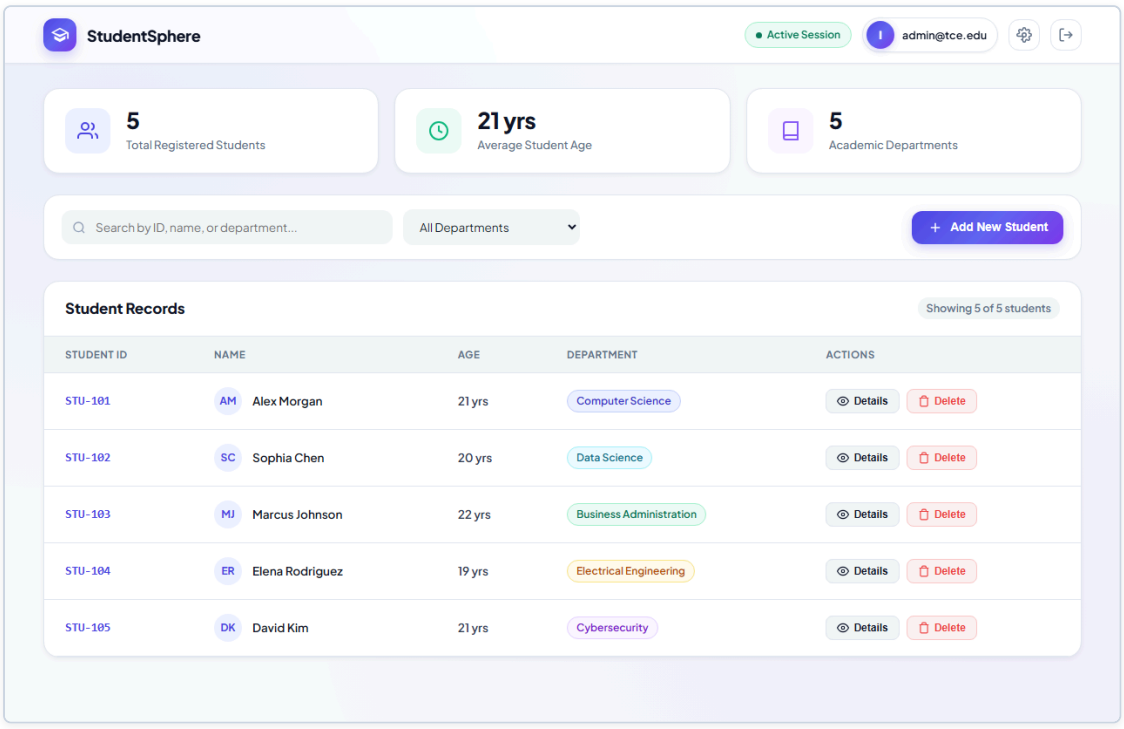


Figure 5: Main Dashboard Displaying Real-Time Student Metrics and Department Roster.

10.4 Firebase Cloud Firestore & Storage Integration

The Firebase Modular SDK (v10.12.0) is configured in `firebase-config.js`. The web client subscribes to student records using Firestore's `onSnapshot` listener, ensuring changes made on any terminal appear instantly:

```
const unsubscribe = onSnapshot(collection(db, "students"), (snapshot) => {
  const students = snapshot.docs.map(doc => doc.data());
  renderStudentTable(students);
  updateKPIs(students);
});
```

Concurrently, the complete roster is written to Google Firebase Storage bucket `website-82bc7.firebasestorage.app` as `students.json`.

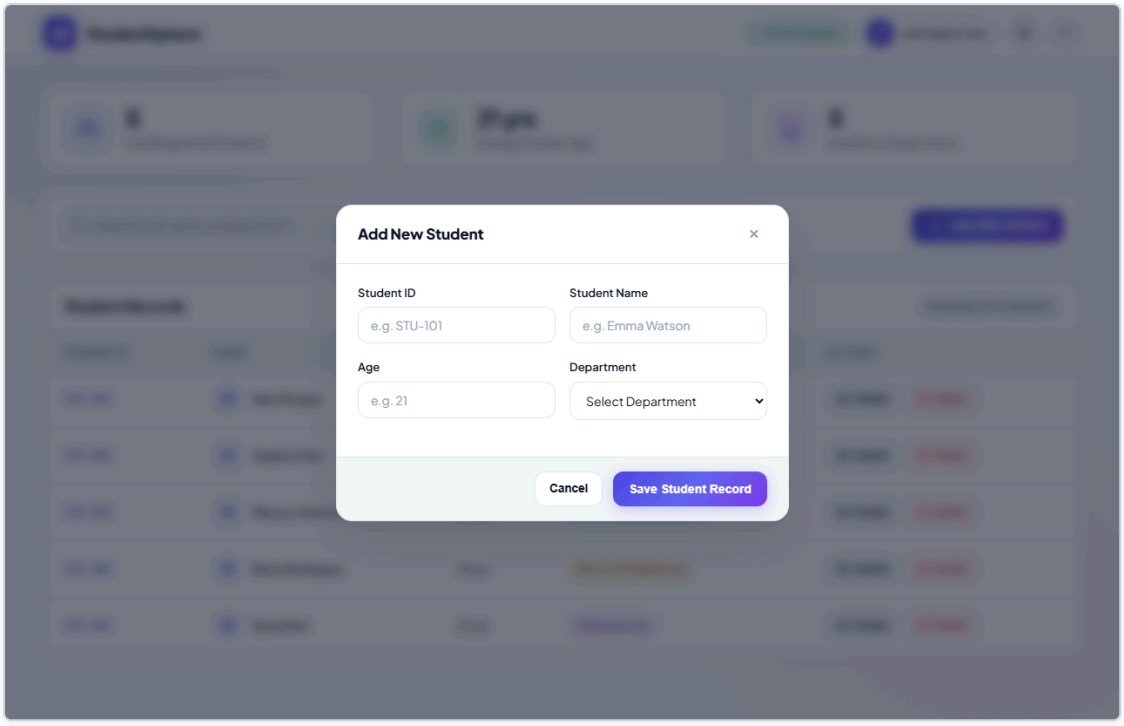


Figure 6: Student Profile Creation Dialog with Field Validation.

10.5 Dropbox API Integration

The Dropbox integration in `dropbox-service.js` communicates directly with the Dropbox Files API v2. A generated OAuth access token is passed in the Authorization header. When student records are updated, the entire student list is serialized as formatted JSON and uploaded to Dropbox:

```
const res = await fetch('https://content.dropboxapi.com/2/files/upload', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${accessToken}`,
    'Dropbox-API-Arg': JSON.stringify({ path: '/students.json', mode: 'overwrite' }),
    'Content-Type': 'application/octet-stream'
  },
  body: new Blob([JSON.stringify(studentsList, null, 2)], { type: 'application/json' })
});
```

The service validates token permissions (`files.content.write` and `files.content.read`), ensuring reliable offsite backup.

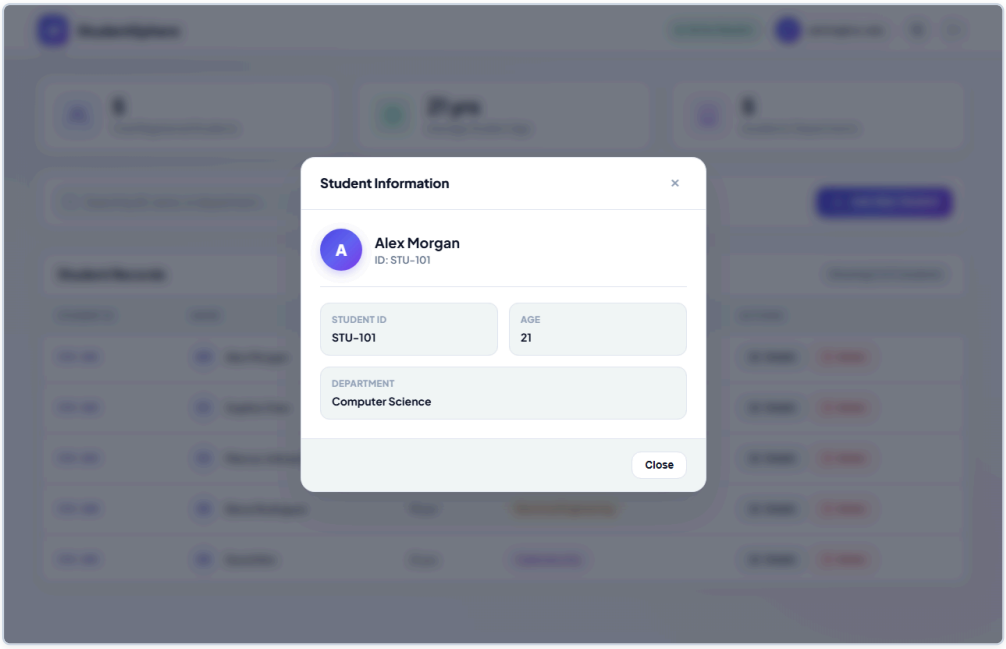


Figure 7: Student Information and Academic Profile Modal.

10.6 Cloud Service Models Used

StudentSphere leverages four primary cloud service models, each fulfilling a specialized role:

Cloud Service	Provider	Cloud Model	Role in StudentSphere
Auth0	Okta	**IDaaS**	Identity management, Universal Login, and Google OAuth SSO.
MongoDB Atlas	MongoDB Inc.	**DBaaS**	Managed NoSQL document database for structured records.
Firebase Firestore	Google Cloud	**BaaS**	Serverless real-time database with instant client stream listeners.
Firebase Storage	Google Cloud	**STaaS**	Cloud object storage bucket for persistent `students.json`.
Dropbox API	Dropbox	**STaaS**	Offsite file storage and automated JSON backup snapshots.

11. Application Screenshots and Results

This section documents the actual user interface and functionality of StudentSphere.

11.1 Authentication & Login Interface

Unauthenticated users are greeted by the authentication screen shown in Figure 4, supporting email/password authentication, registration, and Google OAuth sign-in.

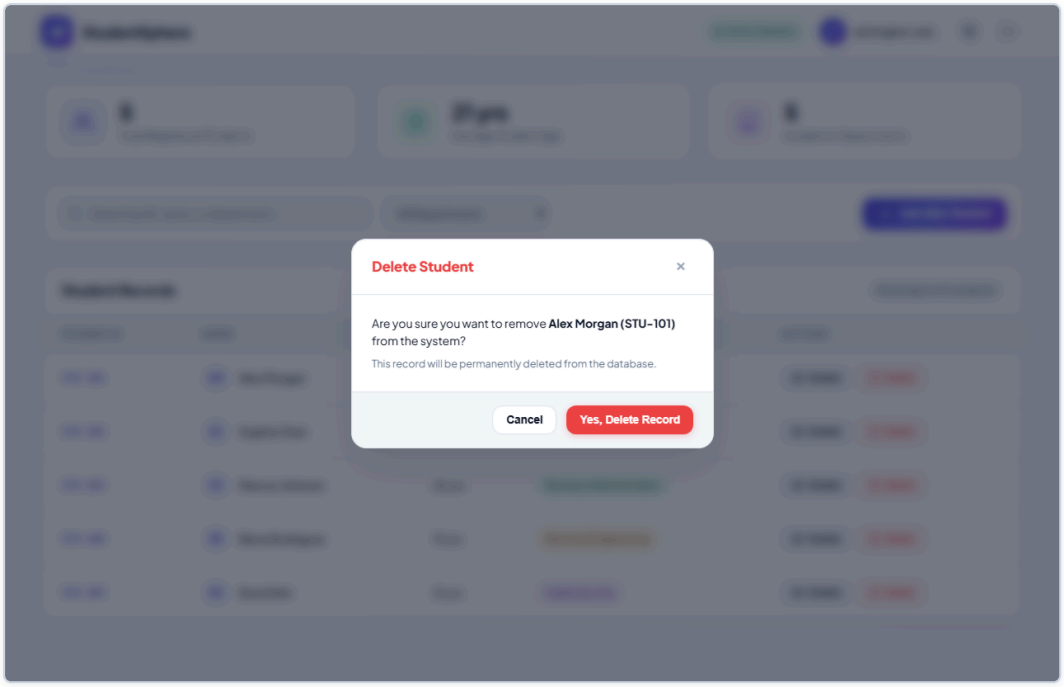


Figure 8: Record Deletion Safety Confirmation Dialog.

11.2 Main Student Dashboard & Metrics

Upon successful login, administrators access the primary dashboard (Figure 5) showing live KPI counters (Total Students, Average Student Age, Total Departments) and the interactive student directory table.

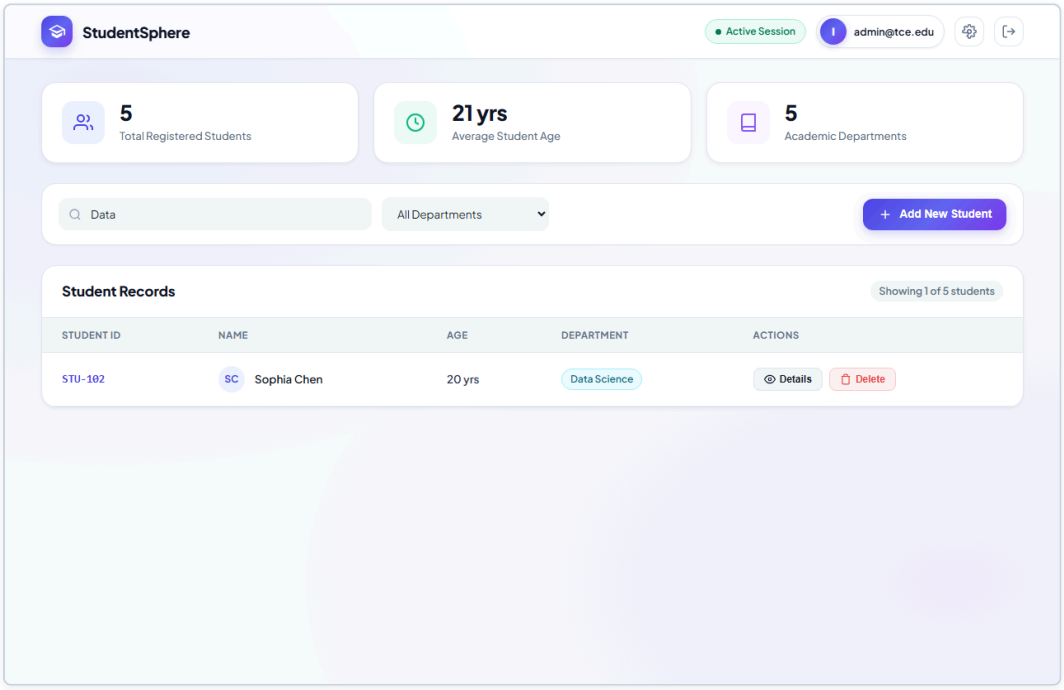


Figure 9: Real-Time Search and Department Filtering Interface.

11.3 Search and Department Filter Operations

The search input filters student records in real time across Student ID, Full Name, and Department. Selecting a department from the dropdown isolates the corresponding roster immediately, as shown in Figure 9.

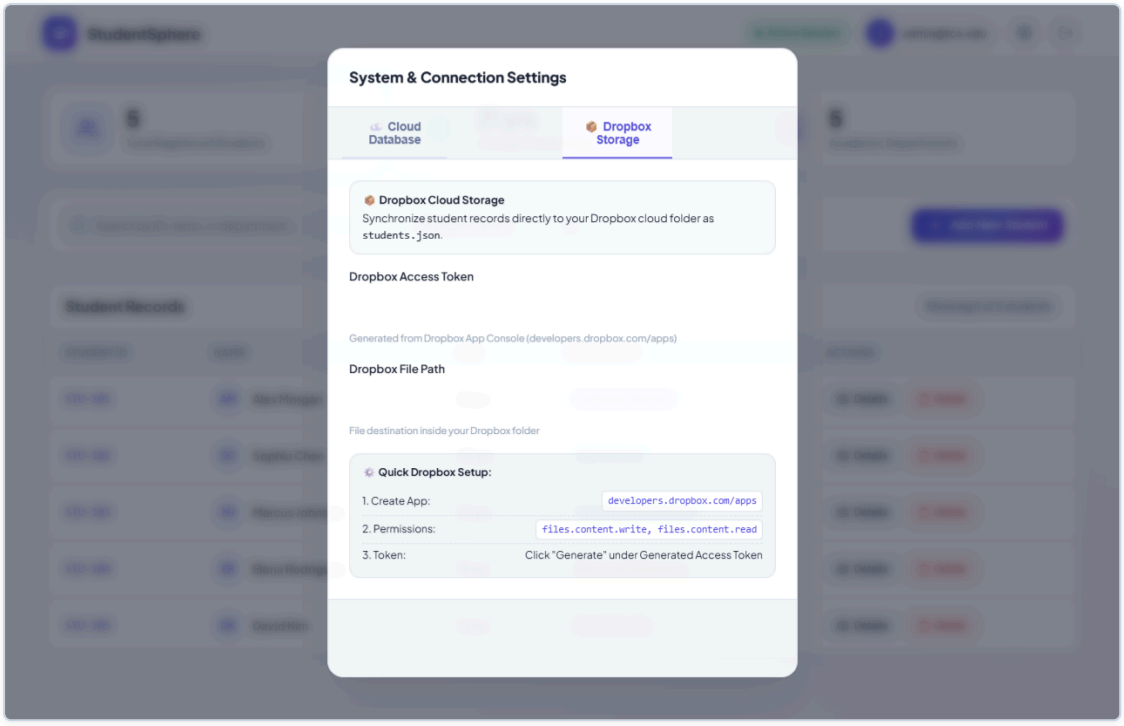


Figure 10: Dropbox Cloud Storage Integration and Token Management.

11.4 Dropbox Cloud Storage Synchronization

The system settings modal includes a dedicated Dropbox tab (Figure 10). Administrators can paste an OAuth access token, verify connection to the Dropbox account, and trigger automated JSON backup uploads to `/students.json`.

11.5 Student Information View and Deletion

Clicking **"Details"** on any student row opens the student inspection modal (Figure 7), while clicking **"Delete"** opens the safety confirmation dialog (Figure 8) to prevent accidental record removal.

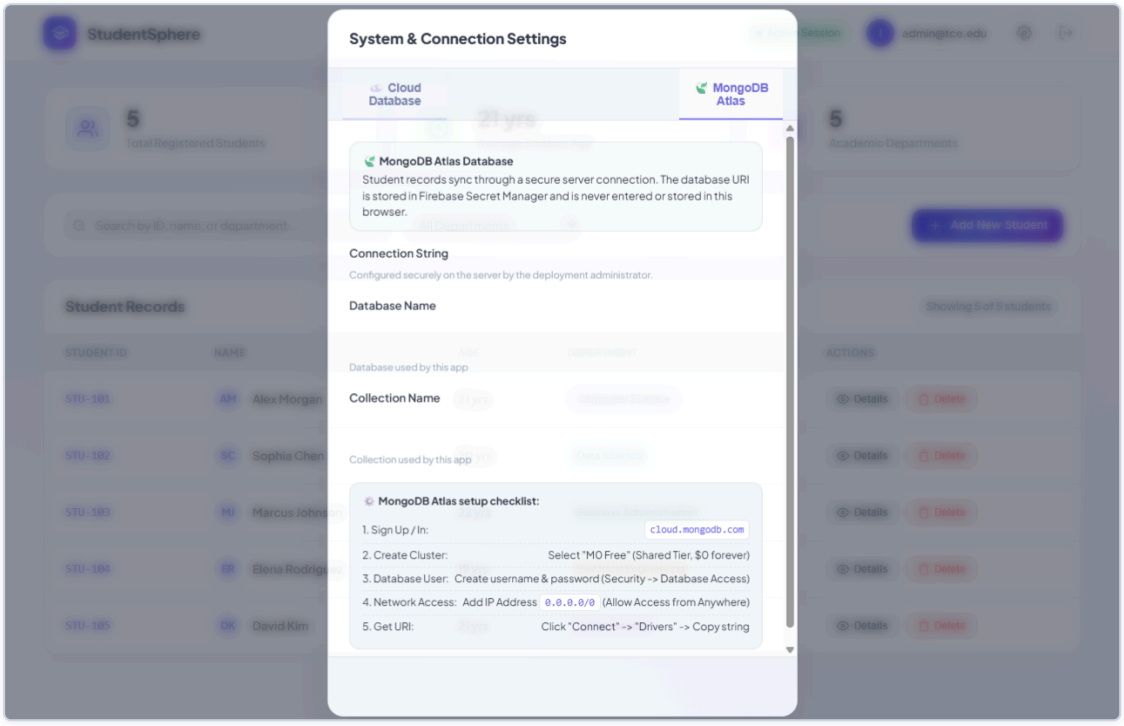


Figure 11: MongoDB Atlas Cloud Database Configuration and Health Diagnostics.

11.6 MongoDB Atlas Database Configuration

The MongoDB Atlas configuration tab (Figure 11) allows administrators to verify cluster connectivity, view database and collection names, and trigger bulk synchronization of student records from the browser to the Atlas cluster.

12. Testing and Results

Testing was conducted to verify application features, authentication flows, database persistence, and multi-cloud synchronization.

Test Case	Expected Result	Actual Result
Start Node.js application server	Server starts on port 3000 without errors	Server running at http://localhost:3000
Sign in with email and password	User is authenticated and redirected to dashboard	Authentication successful; dashboard displayed
Sign in with Google OAuth (Auth0)	Redirects to Google login and returns valid token	Google sign-in completed; session active

Test Case	Expected Result	Actual Result
Real-time KPI calculation	KPI cards calculate total students, average age, and department count accurately	Calculated correctly: 5 students, 21 yrs avg age, 5 departments
Create new student record	Student record is saved and rendered in the table immediately	Student record created; rendered via optimistic update
Firestore real-time synchronization	Records updated in one tab appear instantly in another	Firestore onSnapshot updated secondary client immediately
Upload to Firebase Cloud Storage	JSON array is uploaded to bucket website-82bc7.firebasestorage.app	File students.json updated in Cloud Storage bucket
Live search by student name/ID	Table filters matching student records dynamically	Query 'Data' isolated Data Science student STU-102
Filter by academic department	Table displays only students belonging to selected department	Dropdown selected 'Computer Science'; filtered accurately
View student details	Modal opens displaying complete student profile attributes	Modal opened with ID, name, age, and department
Delete student record	Safety modal prompts confirmation; record is removed on confirm	Record deleted and removed from UI and cloud stores
MongoDB Atlas connection test	POST /api/mongodb/test verifies connectivity to Atlas	Connection ping successful: { success: true }
MongoDB Atlas batch sync	POST /api/mongodb/sync writes complete student roster to Atlas	All student documents synced into studentsphere_db
Dropbox cloud file backup	Student JSON is uploaded to /students.json in Dropbox	File uploaded to Dropbox account successfully
Invalid Dropbox token handling	System catches 401 error and displays friendly toast notification	Toast displayed: 'Invalid or expired Dropbox token'

12.1 Functional Testing

Functional testing confirmed that all primary features—student creation, details viewing, deletion prompts, live search, and department filtering—perform reliably according to specifications.

12.2 Authentication Testing

Authentication testing verified that protected actions require valid session tokens, and that unauthenticated users cannot view or manipulate student records.

12.3 Database Testing

Database testing verified that student documents are correctly created, updated, and deleted in MongoDB Atlas and Google Cloud Firestore. Firestore listeners updated client interfaces without manual refreshes.

12.4 File Storage & Cloud Export Testing

File export testing verified that valid JSON files are generated and uploaded to Google Firebase Storage and Dropbox with correct syntax and write permissions.

12.5 Result Discussion

The application demonstrates how authentication, multi-cloud databases, and cloud file storage can be integrated through a single web portal. During testing, it was noted that MongoDB Atlas free-tier clusters can enter a paused state after inactivity, requiring resumption before SRV connections succeed. Optimistic UI caching ensured that temporary cloud latency never froze the user interface.

13. Advantages

1. **Centralized Access:** A unified administrative workspace for creating, searching, and managing student profiles.
2. **Multi-Cloud Resilience:** Combining Auth0, MongoDB Atlas, Firebase, and Dropbox eliminates single-vendor lock-in.
3. **Zero-Latency UI:** Optimistic caching renders user actions in under 50 milliseconds while background sync occurs asynchronously.
4. **Real-Time Updates:** Firestore listeners update connected clients instantaneously without polling.
5. **Enterprise Authentication:** Auth0 provides secure identity management, ensuring user credentials never touch local servers.
6. **Automated Offsite Backup:** Student rosters are backed up to Dropbox as standardized JSON files for disaster recovery.
7. **Decoupled Architecture:** Frontend and backend services operate independently, simplifying maintenance.

8. **Extensibility:** Additional student attributes, academic grades, and course modules can be added easily.

14. Limitations

- The application depends on internet connectivity for cloud database synchronization.
- Cloud service access depends on valid API tokens, permissions, and network whitelists.
- Free-tier MongoDB Atlas clusters can pause after periods of inactivity and must be resumed before use.
- Dropbox access tokens must be regenerated or refreshed periodically.
- Dropbox synchronization currently pushes files outward but does not parse manual external edits back into the database.
- Cloud service usage is subject to free-tier storage and API call quotas.
- The current implementation focuses on administrative access rather than differentiated multi-tier student/faculty roles.
- The local development proxy setup runs locally and is not automatically accessible over the public internet without hosting deployment.

15. Future Enhancements

1. Implement Role-Based Access Control (RBAC) distinguishing between Student, Faculty, and Admin roles.
2. Add course enrollment tracking, credit hours, and faculty advisor assignments.
3. Provide one-click export of student rosters to Microsoft Excel (.xlsx) and CSV formats.
4. Integrate academic grading scales with automated GPA and CGPA computation.
5. Add student photo upload and identification card storage via Firebase Cloud Storage.
6. Implement bi-directional Dropbox synchronization using webhook notifications.
7. Integrate SMS and email notification services (e.g., Twilio / SendGrid) for institutional announcements.
8. Develop a self-service student portal where students can log in to view their personal academic records.
9. Add dynamic dark/light theme switching with saved user preferences.
10. Package backend services into Docker containers and deploy to Google Cloud Run.
11. Add automated unit and end-to-end testing with GitHub Actions CI/CD pipelines.
12. Implement comprehensive audit logging to track all administrative record modifications.

These are proposed improvements and are not described as completed features.

16. Conclusion

StudentSphere was developed as a cloud computing project to demonstrate the practical integration of identity, database, backend, and storage services in a single academic application.

HTML5 and CSS3 provide an intuitive, responsive user interface, while Node.js and Google Cloud Functions handle backend routing. Auth0 is used for authentication and Google single sign-on, MongoDB Atlas and Firebase Firestore store student records, and Firebase Storage and the Dropbox API store persistent JSON datasets. Together, they demonstrate the IDaaS, DBaaS, BaaS, and STaaS cloud service models.

The project provides practical knowledge of full-stack web development, serverless microservices, token-based authentication, NoSQL database operations, real-time client synchronization, environment configuration, and cloud storage integration.

The implementation can be extended with features such as course enrollment, GPA calculation, Excel exports, and containerized cloud deployment. Overall, the project provides a practical demonstration of cloud computing concepts through a multi-cloud student records and information management platform.

17. References

1. Auth0 Documentation. https://auth0.com/docs
2. Google Cloud Firebase Documentation. https://firebase.google.com/docs
3. Firebase Cloud Firestore Documentation. https://firebase.google.com/docs/firestore
4. Firebase Cloud Storage Documentation. https://firebase.google.com/docs/storage
5. Firebase Admin SDK Documentation. https://firebase.google.com/docs/admin/setup
6. Node.js Official Documentation. https://nodejs.org/docs/latest/api/
7. MongoDB Atlas Documentation. https://www.mongodb.com/docs/atlas/
8. MongoDB Node.js Driver Documentation. https://www.mongodb.com/docs/drivers/node/current/
9. Dropbox API Documentation. https://www.dropbox.com/developers/documentation/http/overview
10. npm Documentation. https://docs.npmjs.com/
11. MDN Web Docs: Modern JavaScript & Fetch API. https://developer.mozilla.org/
12. RFC 6749: The OAuth 2.0 Authorization Framework. https://datatracker.ietf.org/doc/html/rfc6749